AWS Zero To Hero - Day 14

DynamoDB and ElastiCache

Student Guide | Access-pattern-first keys, secondary indexes, capacity modes, TTL, Streams, Global Tables, DAX and cache-engine decisions

Session: Sunday, 16 August 2026, 8:00 PM IST | CloudAdhar x TrainWithShubham

DAY 14 FOCUS

Design DynamoDB from access patterns, then choose indexes, throughput, lifecycle, change capture, multi-Region replication and caching from measurable requirements.

Learning Outcomes

- Translate access patterns into well-distributed partition-key and sort-key designs.
- Distinguish a Global Secondary Index (GSI) from a Local Secondary Index (LSI).
- Choose On-Demand or Provisioned capacity and perform basic RCU/WCU calculations.
- Explain TTL deletion behavior and DynamoDB Streams retention and ordering.
- Select Global Tables for multi-Region active-active requirements.
- Use DAX only when repeated eventually consistent reads justify it.
- Choose Valkey/Redis OSS or Memcached for the correct ElastiCache workload.
- Apply IAM, encryption, private networking, monitoring and cleanup controls.

SAA-C03 Domains and Well-Architected Pillars

Code	Meaning
D1-30%	Domain 1 - Design Secure Architectures (30%)
D2-26%	Domain 2 - Design Resilient Architectures (26%)
D3-24%	Domain 3 - Design High-Performing Architectures (24%)
D4-20%	Domain 4 - Design Cost-Optimized Architectures (20%)
P1	Operational Excellence
P2	Security
P3	Reliability
P4	Performance Efficiency
P5	Cost Optimization
P6	Sustainability

Topics Covered and Core Story

Access-pattern-first keys - D3,D4 | P4,P5 | Best Practice: map frequent reads and writes before creating keys
GSI versus LSI - D3,D4 | P4,P5 | Best Practice: add only indexes required by named queries
Capacity modes - D3,D4 | P4,P5 | Best Practice: match throughput mode to traffic predictability
TTL - D2,D4 | P3,P5 | Best Practice: expire stale items with an asynchronous lifecycle policy
DynamoDB Streams - D2,D3 | P1,P3 | Best Practice: process item changes with idempotent consumers
Global Tables - D2,D3 | P3,P4 | Best Practice: use multi-Region replication for a business requirement
DAX - D3,D4 | P4,P5 | Best Practice: cache repeated eventual reads after fixing the data path
ElastiCache - D2,D3,D4 | P3,P4,P5 | Best Practice: choose rich structures or a simple object cache

REMEMBER THIS

DynamoDB does not begin with entities or joins. It begins with access patterns. A Query needs partition-key equality; the sort key supplies grouping, range and order. A cache cannot repair a poor key design.

Requirement	Select	Why
Unknown or highly variable traffic	On-Demand	No capacity planning; pay per request.
Steady forecastable traffic	Provisioned plus auto scaling	Control throughput and optimize cost.
Query through another partition key	GSI	Independent partition/sort key access path.
Same partition key, alternate sort order	LSI	Alternate sort key; strong reads are possible.
Remove expired sessions or tokens	TTL	Managed asynchronous deletion using epoch seconds.
React to inserts, updates and deletes	DynamoDB Streams	Near-real-time change data capture.
Repeated microsecond DynamoDB reads	DAX	DynamoDB-compatible in-memory acceleration.
Leaderboard, counters or Pub/Sub	Valkey / Redis OSS	Rich data types and replication features.
Simple disposable object cache	Memcached	Simple multi-threaded cache engine.

DOMAIN MAPPING NOTE

Topic-to-domain tags are study guidance. The official SAA-C03 guide publishes domain task statements and weightings rather than an official feature-by-feature mapping.

1. What You Will Learn

Amazon DynamoDB is a serverless NoSQL database designed for predictable performance at scale. AWS manages infrastructure and partition placement. You still own access-pattern discovery, keys, item shape, consistency, indexes, capacity decisions, security, observability and cost controls.

By the end of Day 14, you should be able to explain a DynamoDB design before opening the console.

2. DynamoDB Mental Model

Concept	Meaning	Design consequence
Item	A collection of attributes, up to the service item-size limit	Keep frequently retrieved data together; watch item size.
Partition key	Hash input that determines data distribution	Use high-cardinality values and distribute heavy activity.
Sort key	Orders related items inside a partition-key value	Encode hierarchy, time, type or range conditions.
Composite primary key	Partition key plus sort key	The full pair must be unique.
Query	Reads one partition-key value and optional sort-key range	Preferred access path for known keys.
Scan	Reads every item or index entry before filtering	Reserve for controlled administration or analytics cases.

2.1 Five-step design workflow

1. List every business access pattern in plain language.
2. Write the exact values known at request time.
3. Choose the value that can be tested for equality as the partition key.
4. Use a sort key for grouping, hierarchy, ordering, prefix or range selection.
5. Validate cardinality, item growth, hot-key risk, item size and write amplification.

2.2 Query versus Scan

EXAM SHORTCUT

A FilterExpression does not turn a Scan into an efficient lookup. DynamoDB reads the data first and applies the filter afterward. Use a key condition or an index for a frequent access pattern.

Question	Good answer
Do I know one partition-key value?	Use Query or GetItem if the complete primary key is known.
Do I need an ordered subset?	Use a composite key and a sort-key condition.
Do I only know another attribute?	Design a GSI for the named access pattern.
Do I truly need every item?	Use a controlled Scan, export or analytics design and monitor cost.

3. Access-Pattern-First Key Design

The example below models customer profiles and orders. It intentionally duplicates lookup attributes because DynamoDB optimizes known access paths rather than normalized joins.

3.1 Access-pattern worksheet

Access pattern	Known values	Required result	Frequency
Get customer profile	customerId	One profile item	High
List a customer's orders newest first	customerId	Ordered order headers	High
Get an order by order ID	orderId	One order header	High
List open orders by day	status and date	Operational work queue	Medium

3.2 Candidate physical design

Item type	PK	SK	GSI1PK	GSI1SK
Customer	CUSTOMER#C101	PROFILE	-	-
Order O9001	CUSTOMER#C101	ORDER#2026-08-16T18:30:00Z#O9001	ORDER#O9001	CUSTOMER#C101
Order O9002	CUSTOMER#C101	ORDER#2026-08-15T09:00:00Z#O9002	ORDER#O9002	CUSTOMER#C101

Query the customer's order collection with:

```
PK = CUSTOMER#C101
AND begins_with(SK, ORDER#)
ScanIndexForward = false
```

Query GSI1 to locate an order when only orderId is known:

```
GSI1PK = ORDER#O9001
```

3.3 Sort-key patterns

Pattern	Example	Benefit
Entity type	PROFILE or ORDER#...	Stores several entity types under one partition key.
Time first	ORDER#2026-08-16T18:30:00Z#O9001	Supports chronological range and descending reads.
Hierarchy	IN#KA#BLR	Supports begins_with queries at hierarchy levels.
Version	DOC#D1#v003	Keeps ordered versions near the logical entity.

HOT-KEY WARNING

A key such as STATUS#OPEN can direct every open-order write to one logical key. Add a time bucket or deliberate shard suffix only when the access pattern can query and merge those buckets safely.

4. GSI versus LSI

A secondary index is another maintained access path. Every useful index begins with a named query.

Decision	Global Secondary Index (GSI)	Local Secondary Index (LSI)
Partition key	Can differ from the base table	Must equal the base-table partition key
Sort key	Optional and may differ	Required and differs from the base-table sort key
Creation	Can be added or removed after table creation	Must be defined when the table is created
Read consistency	Eventually consistent only	Eventually or strongly consistent
Capacity	Separate throughput in Provisioned mode	Shares table throughput
Attribute fetch	Cannot fetch non-projected attributes from base table	Can fetch non-projected attributes at extra latency/cost
Item-collection limit	No LSI 10 GB item-collection restriction	10 GB per partition-key value across table and LSIs
Typical use	Query using another partition key	Alternate ordering inside the same partition key

EXAM SHORTCUT

Different partition key: choose a GSI. Same partition key plus an alternate sort key and a strong-read requirement: consider an LSI. In most new designs, a GSI is more flexible.

4.1 Projection choices

Projection	Contains	Use when
KEYS_ONLY	Table keys and index keys	The query needs identifiers and later reads are rare or intentional.
INCLUDE	Keys plus selected non-key attributes	A stable query needs a small known result shape.
ALL	All base-table attributes	The index query usually needs the complete item and storage/write cost is accepted.

4.2 Sparse indexes and write amplification

- An item appears in a GSI only when the required GSI key attributes exist.
- This enables sparse-index patterns such as indexing only escalated orders or unprocessed jobs.
- Each maintained index increases storage and write work; keep the index count purposeful.
- A write can be throttled when a Provisioned GSI does not have enough write capacity.

DEFAULT QUOTAS TO RECOGNIZE

AWS documentation currently lists a default quota of 20 GSIs and a fixed maximum of 5 LSIs per table. Quotas and service capabilities should always be verified for the current account and Region.

5. On-Demand versus Provisioned Capacity

Factor	On-Demand	Provisioned
Management	DynamoDB manages throughput automatically	You define RCU/WCU and may configure auto scaling
Billing	Pay for request units consumed	Pay for capacity provisioned per time period
Best fit	New, unpredictable, intermittent or spiky traffic	Steady, predictable and forecastable traffic
Cost control	Maximum throughput settings can bound usage	Capacity and auto-scaling bounds control throughput
Operational work	Lowest capacity-management effort	Requires monitoring, forecasts and scaling policy review

CURRENT AWS GUIDANCE

On-Demand is the default and AWS-recommended throughput option for most DynamoDB workloads. Provisioned remains useful when traffic is stable and capacity can be forecast reliably.

5.1 Capacity-unit fundamentals

- 1 RCU supports one strongly consistent read per second for an item up to 4 KB.
- 1 RCU supports two eventually consistent reads per second for items up to 4 KB.
- 1 WCU supports one write per second for an item up to 1 KB.
- DynamoDB rounds read size to 4 KB boundaries and write size to 1 KB boundaries.
- Transactional reads and writes consume more capacity than their non-transactional equivalents.

5.2 Practice calculations

Workload	Calculation	Capacity
100 eventually consistent reads/s, 3 KB item	100 x 1 RCU / 2	50 RCU
100 strongly consistent reads/s, 8 KB item	100 x 2 read units	200 RCU
40 writes/s, 1.5 KB item	40 x 2 write units	80 WCU
10 writes/s to table plus one GSI, each index entry <= 1 KB	10 table WCU + 10 GSI WCU	20 WCU total

5.3 Performance and monitoring order

1. Check ThrottledRequests and consumed versus provisioned capacity.
2. Check whether one key is hot before increasing total table capacity.
3. Review item size, consistency, scans, filters and GSI write pressure.
4. Use auto scaling, On-Demand, scheduled scaling or redesigned keys as the evidence requires.

6. Time to Live (TTL)

TTL is a managed way to remove items that are no longer relevant, such as sessions, tokens and temporary events.

Fact	Meaning
Attribute format	Number containing Unix epoch time in seconds.
Deletion time	Asynchronous; expired items are typically deleted within a few days, not at the exact second.
Read behavior	An expired item can remain visible until deletion; filter it in application reads when required.
Capacity	The original service TTL deletion does not consume table write throughput.
Indexes	Deleted items are removed from LSIs and GSIs like ordinary deletes.
Global Tables	Replicated TTL deletes can consume replicated write units in replica Regions.

DO NOT USE TTL AS AN EXACT TIMER

If an action must occur at a precise time, use an event or scheduling design. TTL is a lifecycle and cost-control feature, not a real-time scheduler.

7. DynamoDB Streams

Streams capture a time-ordered sequence of item-level modifications and retain records for up to 24 hours.

Stream view	Record content	Typical use
KEYS_ONLY	Primary key of the changed item	Trigger a lookup or record that a key changed.
NEW_IMAGE	Entire item after the change	Update a search index or derived view.
OLD_IMAGE	Entire item before the change	Archive previous state or inspect deletes.
NEW_AND_OLD_IMAGES	Both versions	Compare fields and build an audit or transition workflow.

- Ordering is preserved for modifications to the same item, not as one global table order.
- A Lambda event source can retry; design the consumer to be idempotent.
- If processing may exceed the 24-hour retention window, design another durable ingestion path.
- A no-op write that does not change item data does not create a stream record.

COMMON PATTERN

DynamoDB table -> Stream -> Lambda -> OpenSearch, S3 archive, notification, materialized view or another service.

8. DynamoDB Global Tables

Global Tables provide managed multi-Region, multi-active replication. Applications use a local Regional replica for low latency and can shift traffic when a Region is impaired.

Decision	MREC	MRSC
Meaning	Multi-Region eventual consistency	Multi-Region strong consistency
Replication	Asynchronous; typically fast but not immediate	A successful write is synchronously replicated to another Region
Reads	Local reads may temporarily be stale	Strongly consistent reads can return the latest successful write
Conflicts	Concurrent writes require conflict awareness; last-writer-wins behavior applies	Strong consistency removes stale-read ambiguity within supported design constraints
TTL	Supported; settings and deletes replicate according to Global Tables behavior	TTL is not supported
Transactions	Atomic only in the Region where invoked; replication is not one cross-Region transaction	DynamoDB transaction operations are not supported
Best fit	Broad multi-Region active-active availability and local performance	Workloads needing multi-Region strong reads and zero-RPO architecture

8.1 Selection checklist

1. Confirm the business needs multi-Region writes; otherwise a simpler recovery design may be enough.
2. Choose consistency mode when creating the global table; do not assume it can be changed later.
3. Plan Regional routing, health checks, failover, IAM, KMS keys and deployment automation.
4. Model replication writes, cross-Region data transfer and replica storage cost.
5. Test application conflict behavior, retry logic and Region-isolation procedures.

DAX AND GLOBAL TABLES

DAX is Regional. Writes replicated into another Region bypass that Region's DAX cache, so cached data may remain stale until its DAX TTL expires. Use short, deliberate cache TTLs and test failover behavior.

8.2 Availability is an application design

A replica alone does not move users. The application needs Regional endpoints, routing policy, dependency readiness, observability and a tested failover runbook. Multi-Region architecture increases both resilience and operational complexity.

9. DynamoDB Performance Decisions and DAX

DynamoDB normally provides single-digit-millisecond performance. DAX targets repeated reads that require microsecond latency.

9.1 Fix the access path before adding a cache

Order	Check	Typical action
1	Access pattern	Replace frequent Scan operations with GetItem, Query or a purposeful index.
2	Key distribution	Remove hot keys or introduce a deliberate sharding/bucketing design.
3	Payload	Reduce item size and project only attributes the request needs.
4	Consistency	Use eventual reads when the business permits them.
5	Request shape	Use BatchGetItem, BatchWriteItem and parallelism carefully where appropriate.
6	Capacity	Select and tune On-Demand or Provisioned capacity from metrics.
7	Cache	Add DAX only when repeated reads and latency targets justify it.

9.2 When DAX is a good fit

- The application performs repeated eventually consistent GetItem, BatchGetItem or Query operations.
- A small group of hot read keys or a high cache-hit rate makes offload valuable.
- Microsecond read latency is a real requirement, not only a preference.
- A Multi-AZ cluster and VPC-based client path are included in the design.

9.3 When DAX is not a good fit

- The application requires strongly consistent reads.
- The workload is write-heavy or reads rarely repeat.
- The expected cache-hit rate is low; AWS guidance notes DAX performs best above 90 percent.
- Normal DynamoDB latency already meets the service-level objective.

DAX CONSISTENCY

DAX serves eventually consistent cached reads. Writes sent through the DAX client are write-through. A writer that bypasses DAX can leave a cached value stale until cache expiration.

Need	DAX	Application-managed cache
DynamoDB API compatibility	Strong fit	Application must own cache-aside/write-through logic
Complex data structures	No	Use Valkey/Redis OSS when required
Cache invalidation control	Managed around DAX client behavior	Explicit application responsibility

10. ElastiCache: Redis versus Memcached

CURRENT ENGINE NAMES

Amazon ElastiCache currently supports Valkey, Redis OSS and Memcached. Certification questions and older designs may say Redis; validate current engine/version support before deployment.

Requirement	Valkey / Redis OSS	Memcached
Data model	Strings plus rich structures such as hashes, lists, sets and sorted sets	Simple key-value objects
Replication and failover	Replication and automatic failover options	No native node replication or automatic failover
Persistence and backup	Backup/restore and durability features depend on engine and deployment	Treat node-based cache data as disposable
Pub/Sub	Supported	Not supported
Leaderboards and counters	Strong fit	Not the primary fit
Engine threading	Traditionally single-threaded command execution with evolving I/O improvements	Multi-threaded
Horizontal partitioning	Cluster mode supports sharding	Client-side distribution across nodes
Best use	Sessions, rate limiting, queues, counters, leaderboards, geospatial or rich cache logic	Large simple object cache with disposable values

10.1 DAX versus ElastiCache

Requirement	Choose
DynamoDB-compatible, repeated eventually consistent reads	DAX
Leaderboard, distributed counter, rate limiter or Pub/Sub	Valkey / Redis OSS
Simple disposable object fragments with a minimal model	Memcached
Strongly consistent DynamoDB read	Read DynamoDB directly; do not route it through DAX

10.2 Security pattern

- Place DAX and ElastiCache inside approved VPC subnets; they are not public web services.
- Allow the cache port only from the application security group.
- Enable supported encryption in transit and at rest, authentication and least-privilege access.
- Store tokens and credentials in an approved secret store; never embed them in repositories or screenshots.
- Monitor evictions, memory, connections, CPU, replication lag and cache-hit ratio.

11. Student Practical - Access Patterns

This practical uses synthetic data. Console labels and Regions can change. Use an instructor-approved account, review costs and do not create DAX or ElastiCache clusters unless the instructor explicitly authorizes them.

11.1 Resource plan

Resource	Suggested value
DynamoDB table	cloudadhar-orders-day14
Partition key	PK (String)
Sort key	SK (String)
GSI name	GSI1
GSI partition key	GSI1PK (String)
GSI sort key	GSI1SK (String)
Capacity mode	On-Demand
TTL attribute	ExpiresAt
Stream view	NEW_AND_OLD_IMAGES

Use tags such as Project=AWS-Zero-To-Hero, Day=14, Environment=Training, Owner=CloudAdhar and a current cleanup date.

11.2 Create the table

1. Open Amazon DynamoDB > Tables > Create table.
2. Enter table name cloudadhar-orders-day14.
3. Set partition key PK to String and sort key SK to String.
4. Choose Customize settings so the capacity and index choices are visible.
5. Choose On-Demand capacity for this small unpredictable training workload.
6. Create GSI1 with GSI1PK as the partition key and GSI1SK as the sort key.
7. For the lab, choose ALL projection so the GSI query returns the full synthetic order.
8. Keep encryption enabled. Use an approved KMS option required by the training account.
9. Create the table and wait until Table status and GSI status are Active.

11.3 Minimum IAM permissions for the learner role

Use a constrained training role that permits the required DynamoDB actions on the Day 14 table only. Typical actions include CreateTable, DescribeTable, PutItem, GetItem, Query, UpdateTable, UpdateTimeToLive, DescribeTimeToLive and DeleteTable. Do not use an administrator identity when a scoped role is available.

SECURITY CHECK

DynamoDB access is controlled with IAM. A DynamoDB gateway VPC endpoint can keep supported VPC traffic on the AWS network and can apply an endpoint policy.

11.4 Insert synthetic items with AWS CloudShell

Set the Region and use only non-sensitive training data.

```
TABLE=cloudadhar-orders-day14
REGION=<approved-region>
```

Insert the customer profile:

```
aws dynamodb put-item --region "$REGION" --table-name "$TABLE" --item '{
  "PK": {"S": "CUSTOMER#C101"},
  "SK": {"S": "PROFILE"},
  "Name": {"S": "Asha Student"}
}'
```

Insert two order headers:

```
aws dynamodb put-item --region "$REGION" --table-name "$TABLE" --item '{
  "PK": {"S": "CUSTOMER#C101"},
  "SK": {"S": "ORDER#2026-08-16T18:30:00Z#O9001"},
  "GSI1PK": {"S": "ORDER#O9001"},
  "GSI1SK": {"S": "CUSTOMER#C101"},
  "Status": {"S": "PAID"}, "Total": {"N": "2499"}
}'

aws dynamodb put-item --region "$REGION" --table-name "$TABLE" --item '{
  "PK": {"S": "CUSTOMER#C101"},
  "SK": {"S": "ORDER#2026-08-15T09:00:00Z#O9002"},
  "GSI1PK": {"S": "ORDER#O9002"},
  "GSI1SK": {"S": "CUSTOMER#C101"},
  "Status": {"S": "OPEN"}, "Total": {"N": "899"}
}'
```

11.5 Query the base table

```
aws dynamodb query --region "$REGION" --table-name "$TABLE" \
  --key-condition-expression 'PK = :pk AND begins_with(SK, :prefix)' \
  --expression-attribute-values '{
    ":pk": {"S": "CUSTOMER#C101"},
    ":prefix": {"S": "ORDER#"}
  }' --no-scan-index-forward
```

Expected evidence: two order items in newest-first sort-key order. No Scan is required.

11.6 Query the GSI

```
aws dynamodb query --region "$REGION" --table-name "$TABLE" \
  --index-name GSI1 \
  --key-condition-expression 'GSI1PK = :order' \
  --expression-attribute-values '{
    ":order": {"S": "ORDER#O9001"}
  }'
```

Expected evidence: order O9001 is returned using orderId without knowing customerId.

12. TTL and Streams Practical

12.1 Create an expiring synthetic session

```
EXPIRES_AT=$(date -u -d '+2 hours' +%s)
aws dynamodb put-item --region "$REGION" --table-name "$TABLE" --item "{
  \"PK\": {\"S\": \"SESSION#S1001\"},
  \"SK\": {\"S\": \"META\"},
  \"ExpiresAt\": {\"N\": \"$EXPIRES_AT\"}
}"

aws dynamodb update-time-to-live --region "$REGION" --table-name "$TABLE" \
  --time-to-live-specification 'Enabled=true,AttributeName=ExpiresAt'
```

Confirm with DescribeTimeToLive. Do not wait for exact-time deletion; TTL is asynchronous.

12.2 Enable the stream

```
aws dynamodb update-table --region "$REGION" --table-name "$TABLE" \
  --stream-specification StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES

aws dynamodb describe-table --region "$REGION" --table-name "$TABLE" \
  --query 'Table.LatestStreamArn'
```

Optional instructor extension: connect a Lambda consumer, make one item change and inspect the event. The function must be idempotent.

12.3 Troubleshooting order

Symptom	Check first
ValidationException on Query	Partition-key equality, attribute names/types and expression placeholders.
GSI returns no item	GSI is Active; item contains both required GSI key attributes; allow for eventual consistency.
AccessDeniedException	Learner-role policy, table ARN, GSI ARN and Region.
Throttling	Hot-key distribution, capacity mode, GSI capacity and item size.
Expired item remains	TTL status, Number type, epoch seconds and asynchronous deletion behavior.
Lambda processes an event again	Idempotency key, partial-batch failure handling and retry behavior.

12.4 Cleanup

1. Capture only non-sensitive evidence required by the instructor.
2. Delete any optional Lambda event source mapping and Lambda function.
3. Delete cloudadhar-orders-day14 after confirming no reusable training data is needed.
4. If the instructor approved DAX or ElastiCache, delete the cluster and its training-only resources.
5. Remove alarms, log groups, VPC endpoints or security groups created only for this lab.
6. Confirm the Billing console has no unexpected active resources.

COST WARNING

DAX and node-based ElastiCache resources are billed while running. A design discussion is sufficient for this session unless the instructor approves a timed practical and verifies cleanup.

13. SAA-C03 Decision Practice

Scenario	Best decision	Reason
New API has unpredictable request volume	DynamoDB On-Demand	Avoid initial capacity forecasting.
Steady 24x7 workload with reliable history	Provisioned plus auto scaling	Tune cost and capacity to forecasts.
Find an order when only orderId is known	GSI on orderId	Creates the required alternate partition-key access path.
Same customer, orders sorted by another attribute with strong reads	LSI if planned at table creation	Same partition key and alternate sort key.
Remove sessions after expiry	TTL	Managed asynchronous lifecycle deletion.
Send changed items to a processing function	DynamoDB Streams plus Lambda	Near-real-time change capture.
Active-active reads and writes in several Regions	Global Tables	Managed multi-Region replication.
Repeated eventually consistent product reads need microseconds	DAX	DynamoDB-compatible read cache.
Gaming leaderboard and atomic counters	Valkey / Redis OSS	Sorted sets and rich operations.
Simple disposable rendered-page cache	Memcached	Minimal multi-threaded object cache.

14. One-Minute Recap

- Start with access patterns. Query needs partition-key equality.
- The sort key provides order, prefix, hierarchy and range behavior.
- GSI means another partition key; LSI means the same partition key and another sort key.
- On-Demand favors unknown traffic; Provisioned favors predictable traffic.
- TTL is asynchronous. Streams retain item-change records for 24 hours.
- Global Tables provide multi-Region, multi-active replication.
- DAX is for repeated eventually consistent DynamoDB reads, not strong reads.
- Valkey/Redis OSS is feature-rich; Memcached is a simple disposable object cache.

15. Official AWS References

Partition-key design: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key-design.html

Secondary-index guidance: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-indexes-general.html

DynamoDB capacity modes: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/capacity-mode.html

DynamoDB TTL: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html

DynamoDB Streams: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html

Global Tables: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GlobalTables.html

DynamoDB Accelerator (DAX): https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/DAX.html

ElastiCache engine comparison: https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/SelectEngine.html